Project Report

Aditya Vegad

March 2, 2025

Group Members

- Vegad Aditya Rakeshbhai - 2023CS10819
- Vishal Kumar - 2023CS10816
- Sourav Kumar Patel - 2023CS10751

1 Design Decisions

- **Parsing the Input:**
  - The input is parsed into different parts: type of instruction, target cell, and parameters.
  - These are stored in a structure called `command_call`.
- **Cell Structure:**
  - A structure called `cell` is used to represent a spreadsheet cell.
  - It contains:
    - The cell value.
    - A hashset to store dependencies (other cells that depend on this cell).
    - A `command_call` instance.
- **Why HashSet?**
  - If a formula in a cell is updated, old dependencies must be removed, and new dependencies must be added.
  - The hashset allows us to optimize time as well as space depending upon the need by changing size of buckets.
- **Determining Calculation Order:**
  - A **directed acyclic graph (DAG)** is created for cells.
  - **Topological sorting** is used to determine the order of instructions to avoid unnecessary recalculations.
  - The instructions are calculated in **reverse topological order** (i.e., instructions affecting the least are calculated first).
- **Handling Errors:**
  - **Circular Dependency Check:** If a circular dependency is found, an error is raised.
  - **Division by Zero Check:**
    - A unit bit is assigned to each cell.
    - It is set to **1** if a division by zero occurs.
- **Updating Cell Values:**
  - After getting the sorted order, a function named `update` is called.
  - The `update` function processes the calculations for functions and arithmetic in **reverse topological order**.

2 Challenges Faced

- **Memory Optimization:**
  - Initially, everything was string-based (e.g., hashset and parameters).
  - Converted strings to integers where the first three digits represent the row and the last five digits represent the column.
  - Optimized instruction command type into opcode using bitfields.
  - However, memory usage was still high due to padding issues, which were resolved by minimizing data structure padding.
- **Parsing Complex Inputs:**
  - Handling expressions like `(+5--3)` was difficult due to multiple consecutive symbols.
  - Developed a more robust parser to handle such cases correctly.
- **Circular Dependency and Topological Sorting:**
  - Checking for circular dependencies and performing topological sorting.
  - The initial implementation was recursive, which could cause a stack limit error.
  - Optimized by using an iterative approach to prevent excessive recursion depth.

3 Structure of the Program

- **Parsing the Input:**
  - The input is parsed into different parts: type of instruction and parameters.
  - These are stored in a structure called `command_call`.
- **Topological Sorting:**
  - After parsing, we determine the order of calculations using **topological sorting**.
  - This ensures that dependencies are resolved before computation.
- **Updating Cell Values:**
  - A function named `update` is called to update each cell based on the instruction.
  - Different functions such as `max`, `avg`, `sum`, `stdev`, etc., are used for calculations.
- **Handling Circular Dependencies:**
  - If a circular dependency is detected, we undo the operations performed so far.
  - This prevents inconsistent states in the spreadsheet.
- **Processing New Instructions:**
  - Once the previous calculations are complete, the system is ready to process new instructions.

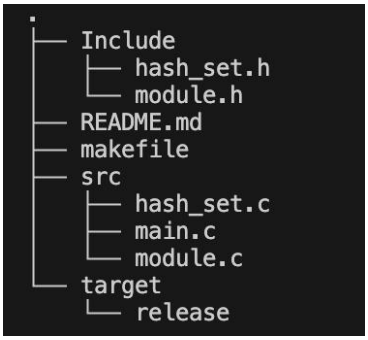


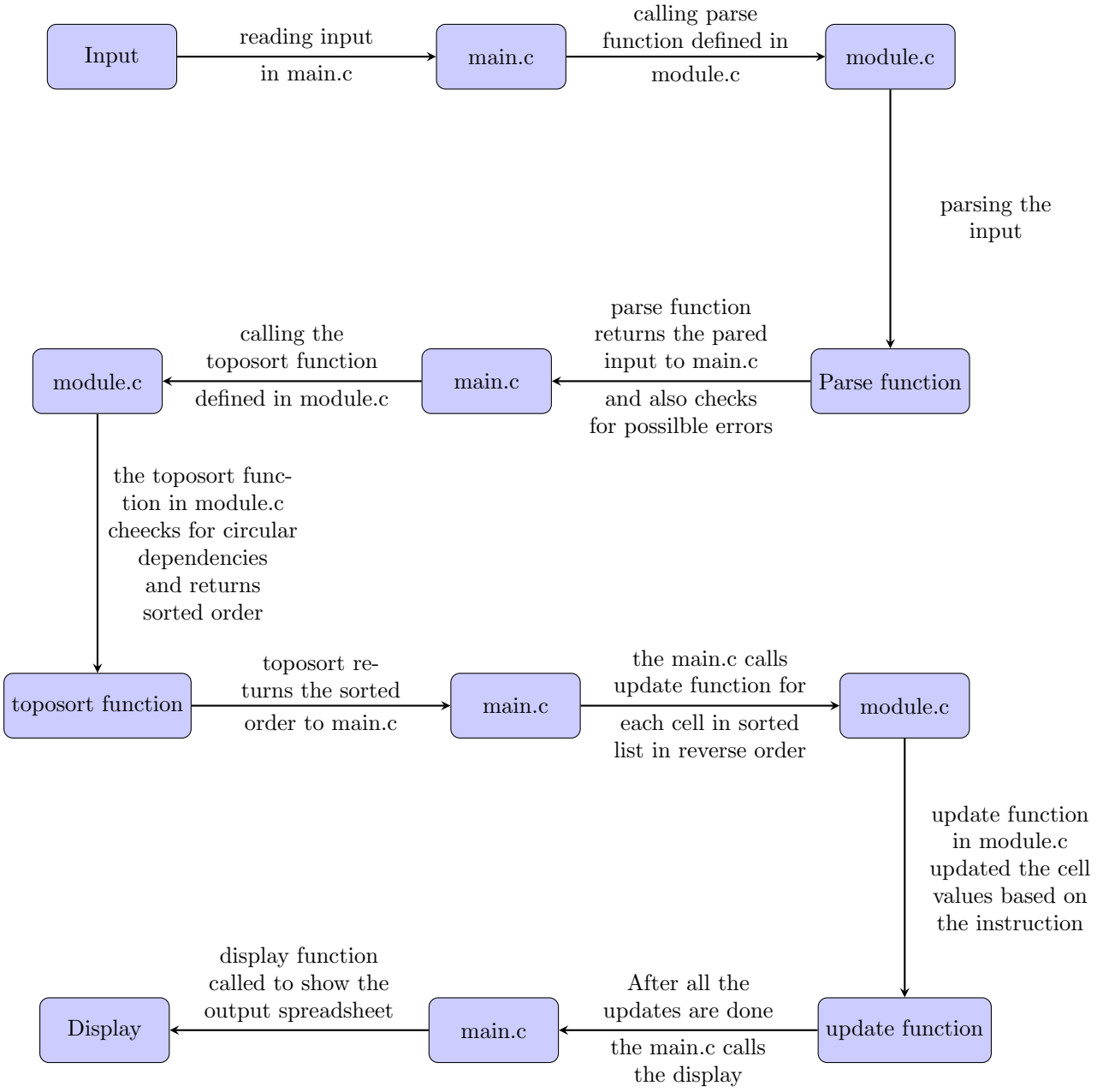
Figure 1: Structure of program.

4 Edge Cases

Describe the edge cases considered and how they were handled.

- **Empty Input:**
  - Empty string does not match with any of the regex patterns hence gives Invalid Input.
- **Circular Dependencies:**
  - When we give circular dependencies as input, the program detects it by using topological sort and states "Circular Dependency".The old values are retained.
- **Division by Zero:**
  - The program checks for division by zero and sets a specific flag to handle it gracefully. This value is displayed as "ERR".
- **White space:**
  - The program removes all white spaces by iterating through the string and removing them.
- **Duplicate in hashset:**
  - The program checks for previous occurance of the cell in the hashset and does not add it again.
- **Scroll to corner cells:**
  - The program scrolls to the corner cells when scroll_to cell is given as input. If 10 more cells are not available in each direction then it only shows the available cells.
- **1 x 1 sized spreadsheet:**
  - The program handles the case when the spreadsheet is of size 1 x 1 without any errors by displaying the single cell value.
- **Out of range cell :**
  - If the input contains any cell which is in the range of the spreadsheet then "Invalid" status is given.

5 Program Workflow



6 Upload Links

Provide links to important uploads (e.g., GitHub repository, documentation, final submission).
GitHub Repository Video Link